

优化程序性能

计算机系统基础

任课教师:

龚奕利

yiligong@whu.edu.cn

主要内容

- 概述
- 通用的优化方法
 - 代码移动/预计算
 - 强度消减
 - 共享公共子表达式
 - 删除不必要的过程调用
- 妨碍优化的因素
 - 过程调用
 - 内存别名
- 利用指令级并行
- 处理条件分支

关于性能的事实

- 性能不仅仅是渐近复杂度（大O）
- 常数因子也很重要！
 - 根据代码的编写方式，性能可能有 **10:1** 的差距
 - 必须在多个层面进行优化：
 - 算法，数据表示，过程，循环
- 必须了解系统以优化性能
 - 程序如何被编译和执行
 - 现代处理器和内存系统如何运行
 - 如何衡量程序性能并识别瓶颈
 - 如何在不破坏代码模块性和通用性的前提下提升性能

优化编译器

- 提供程序到机器码的高效映射
 - 寄存器分配
 - 代码选择和排序（调度）
 - 死代码删除
 - 消除细微的低效部分
- 不（经常）提高渐近效率
 - 需要程序员选择整体上最佳的算法
 - 大 O 的优化通常比常数因子更重要
 - 但常数因子也很重要
- 难以克服“妨碍优化的因素”
 - 可能的内存别名
 - 可能的过程副作用

优化编译器的局限性

■ 在基本约束下运行

- 不能引起程序行为的任何变化
 - 除非程序使用了语言的非标准特性
- 这通常会阻止编译器进行某些仅在极端情况下影响行为的优化

■ 在程序员看来显而易见的行为可能被语言和编码风格掩盖

- 例如，数据范围通常比变量类型暗示的更小

■ 大多数分析仅在过程内部进行

- 整个程序的分析在大多数情况下过于昂贵
- 较新的 GCC 版本支持单个文件内的跨过程分析
 - 但不同文件之间的代码不进行此类分析

■ 大多数分析仅基于静态信息

- 编译器难以预测运行时的输入

■ 存疑时，编译器必须保持保守

```
1 void twiddle1(long *xp, long *yp)
2 {
3     *xp += *yp;
4     *xp += *yp;
5 }
6
7 void twiddle2(long *xp, long *yp)
8 {
9     *xp += 2* *yp;
10 }
```

通用的优化方法

- 无论处理器或编译器如何，程序员或编译器都应执行的优化
- 代码移动
 - 减少重复计算的频率
 - 如果计算结果始终相同
 - 尤其是将代码移出循环

```
void set_row(double *a, double *b,  
             long i, long n)  
{  
    long j;  
    for (j = 0; j < n; j++)  
        a[n*i+j] = b[j];  
}
```



```
long j;  
int ni = n*i;  
for (j = 0; j < n; j++)  
    a[ni+j] = b[j];
```

编译器生成的代码移动(-01)

```
void set_row(double *a, double *b,  
            long i, long n)  
{  
    long j;  
    for (j = 0; j < n; j++)  
        a[n*i+j] = b[j];  
}
```

```
long j;  
long ni = n*i;  
double *rowp = a+ni;  
for (j = 0; j < n; j++)  
    *rowp++ = b[j];
```

```
set_row:  
    testq    %rcx, %rcx          # Test n  
    jle      .L1                 # If 0, goto done  
    imulq    %rcx, %rdx          # ni = n*i  
    leaq     (%rdi,%rdx,8), %rdx  # rowp = A + ni*8  
    movl     $0, %eax            # j = 0  
    .L3:                                     # loop:  
    movsd    (%rsi,%rax,8), %xmm0 # t = b[j]  
    movsd    %xmm0, (%rdx,%rax,8) # M[A+ni*8 + j*8] = t  
    addq     $1, %rax            # j++  
    cmpq     %rcx, %rax          # j:n  
    jne      .L3                 # if !=, goto loop  
    .L1:                                     # done:  
    rep ; ret
```


强度消减

- 用简单操作替代高成本操作
- 使用移位、加法替代乘法或除法

$16 * x \rightarrow x \ll 4$

- 效果依赖于机器架构
- 取决于乘法或除法指令的成本

- 在 Intel Nehalem 上, 整数乘法需要 3 个 CPU 周期

- 识别乘积序列

```
for (i = 0; i < n; i++) {  
    int ni = n*i;  
    for (j = 0; j < n; j++)  
        a[ni + j] = b[j];  
}
```



```
int ni = 0;  
for (i = 0; i < n; i++) {  
    for (j = 0; j < n; j++)  
        a[ni + j] = b[j];  
    ni += n;  
}
```

共享公共子表达式

- 重复利用公共子表达式的计算结果
- GCC 在 -O1 时执行

```
/* Sum neighbors of i,j */
up =    val[(i-1)*n + j ];
down =  val[(i+1)*n + j ];
left =  val[i*n      + j-1];
right = val[i*n      + j+1];
sum = up + down + left + right;
```

3 multiplications: $i*n$, $(i-1)*n$, $(i+1)*n$

```
leaq    1(%rsi), %rax    # i+1
leaq    -1(%rsi), %r8    # i-1
imulq   %rcx, %rsi      # i*n
imulq   %rcx, %rax      # (i+1)*n
imulq   %rcx, %r8       # (i-1)*n
addq    %rdx, %rsi      # i*n+j
addq    %rdx, %rax      # (i+1)*n+j
addq    %rdx, %r8       # (i-1)*n+j
```

```
long inj = i*n + j;
up =    val[inj - n];
down =  val[inj + n];
left =  val[inj - 1];
right = val[inj + 1];
sum = up + down + left + right;
```

1 multiplication: $i*n$

```
imulq   %rcx, %rsi      # i*n
addq    %rdx, %rsi      # i*n+j
movq    %rsi, %rax      # i*n+j
subq    %rcx, %rax      # i*n+j-n
leaq    (%rsi,%rcx), %rcx # i*n+j+n
```

妨碍优化的因素 #1: 过程调用

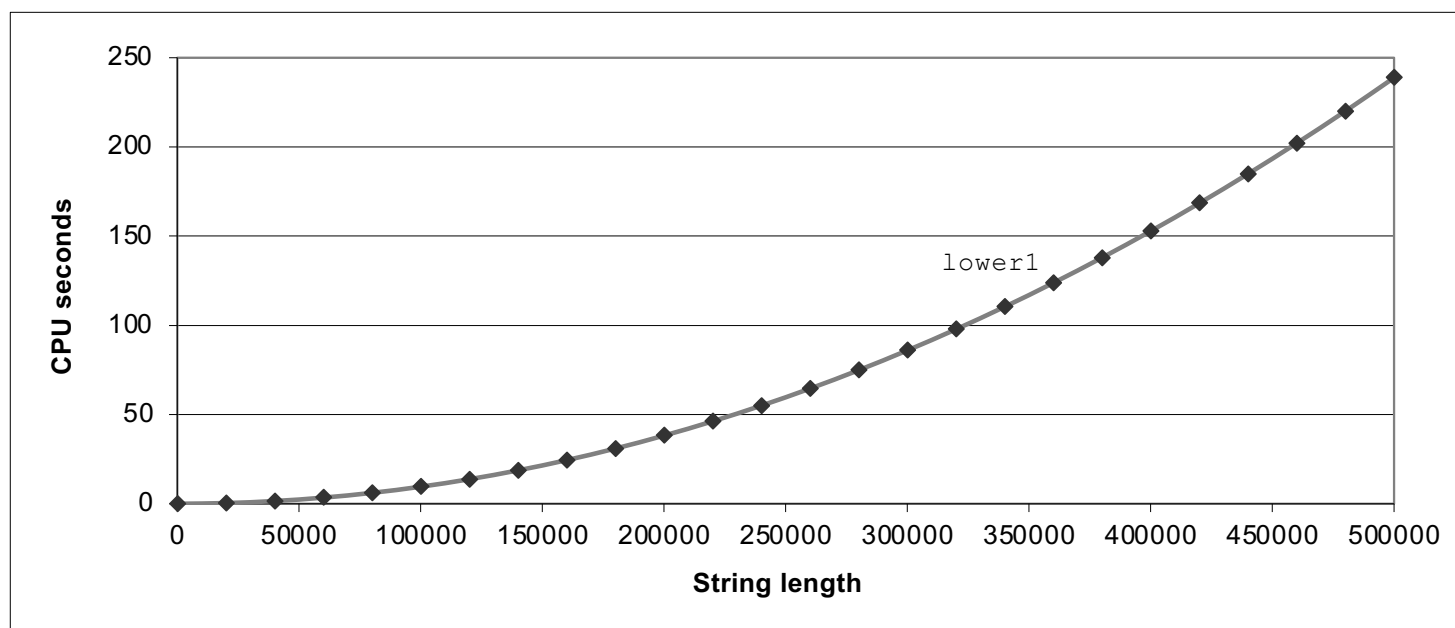
■ 将字符串转换为小写的过程

```
void lower(char *s)
{
    size_t i;
    for (i = 0; i < strlen(s); i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}
```

0	00	NUL	25	19	EM	51	33	3	77	4D	M	103	67	g
1	01	SOH	26	1A	SUB	52	34	4	78	4E	N	104	68	h
2	02	STX	27	1B	ESC	53	35	5	79	4F	O	105	69	i
3	03	ETX	28	1C	FS	54	36	6	80	50	P	106	6A	j
4	04	EOT	29	1D	GS	55	37	7	81	51	Q	107	6B	k
5	05	ENQ	30	1E	RS	56	38	8	82	52	R	108	6C	l
6	06	ACK	31	1F	US	57	39	9	83	53	S	109	6D	m
7	07	BEL	32	20	space	58	3A	:	84	54	T	110	6E	n
8	08	BS	33	21	!	59	3B	;	85	55	U	111	6F	o
9	09	HT	34	22	"	60	3C	<	86	56	V	112	70	p
10	0A	LF	35	23	#	61	3D	=	87	57	W	113	71	q
11	0B	VT	36	24	\$	62	3E	>	88	58	X	114	72	r
12	0C	FF	37	25	%	63	3F	?	89	59	Y	115	73	s
13	0D	CR	38	26	&	64	40	@	90	5A	Z	116	74	t
14	0E	SO	39	27	'	65	41	A	91	5B	[	117	75	u
15	0F	SI	40	28	(	66	42	B	92	5C	\	118	76	v
16	10	DLE	41	29	)	67	43	C	93	5D	]	119	77	w
17	11	DC1	42	2A	*	68	44	D	94	5E	^	120	78	x
18	12	DC2	43	2B	+	69	45	E	95	5F	_	121	79	y
19	13	DC3	44	2C	,	70	46	F	96	60	`	122	7A	z
20	14	DC4	45	2D	-	71	47	G	97	61	a	123	7B	{
21	15	NAK	46	2E	.	72	48	H	98	62	b	124	7C	
22	16	SYN	47	2F	/	73	49	I	99	63	c	125	7D	}
23	17	ETB	48	30	0	74	4A	J	100	64	d	126	7E	~
24	18	CAN	49	31	1	75	4B	K	101	65	e	127	7F	DEL
			50	32	2	76	4C	L	102	66	f			

小写转换程序的性能

- 当字符串长度加倍时，所需时间变为四倍
- 性能呈二次曲线



将循环转换为 **Goto** 代码

```
void lower(char *s)
{
    size_t i = 0;
    if (i >= strlen(s))
        goto done;
loop:
    if (s[i] >= 'A' && s[i] <= 'Z')
        s[i] -= ('A' - 'a');
    i++;
    if (i < strlen(s))
        goto loop;
done:
}
```

- `strlen` 每次循环都会被调用

调用 Strlen

```
/* My version of strlen */
size_t strlen(const char *s)
{
    size_t length = 0;
    while (*s != '\0') {
        s++;
        length++;
    }
    return length;
}
```

■ Strlen 的性能

- 确定字符串长度的唯一方法是扫描整个字符串，寻找空字符 `\0`

■ 整体性能分析：对于长度为 `N` 的字符串

- 调用 `N` 次 `strlen`
- 每次需要的时间为 `N`
- 整体时间复杂度为 $O(N^2)$

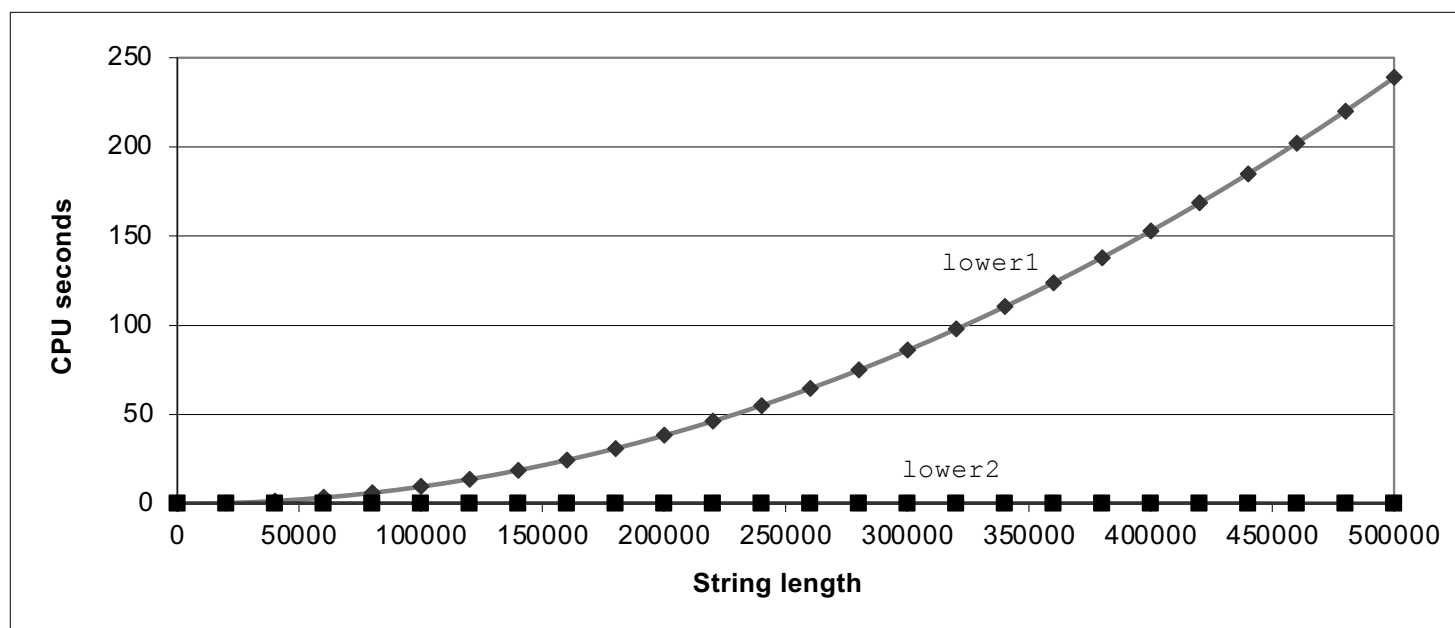
提高性能

```
void lower(char *s)
{
    size_t i;
    size_t len = strlen(s);
    for (i = 0; i < len; i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}
```

- 将 strlen 的调用移出循环
- 因为每次循环迭代中字符串长度都不会改变
- 这是代码移动的一种形式

小写转换程序的性能

- 当字符串长度加倍时，所需时间变为两倍
- `lower2` 性能呈线性曲线



妨碍优化的因素：过程调用

■ 为什么编译器不把 `strlen` 移出循环？

- 过程可能有副作用
 - 每次调用修改全局变量
- 函数对相同参数的返回结果可能不同
 - 依赖某些全局变量
 - `lower` 可能与 `strlen` 交互

■ 注意：

- 编译器将过程调用视为黑箱
- 在这些调用附近的优化较弱

■ 补救方法：

- 使用内联函数
 - GCC 在 `-O1` 时执行
 - 在单个文件内
- 手动进行代码移动

```
size_t lencnt = 0;
size_t strlen(const char *s)
{
    size_t length = 0;
    while (*s != '\0') {
        s++; length++;
    }
    lencnt += length;
    return length;
}
```

```
#ifdef __cplusplus
inline int my_func(int x) { ... }
#else
static inline int my_func(int x) { ... }
#endif
```

内存很重要

```
/* Sum rows is of n X n matrix a
   and store in vector b */
void sum_rows1(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += a[i*n + j];
    }
}
```

```
# sum_rows1 inner loop
.L4:
    movsd    (%rsi,%rax,8), %xmm0    # FP load
    addsd    (%rdi), %xmm0           # FP add
    movsd    %xmm0, (%rsi,%rax,8)    # FP store
    addq     $8, %rdi
    cmpq     %rcx, %rdi
    jne      .L4
```

- 代码在每次循环中更新 b[i]
- 为什么编译器不能优化这个操作？

内存别名

```
/* Sum rows is of n X n matrix a
   and store in vector b */
void sum_rows1(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += a[i*n + j];
    }
}
```

```
double A[9] =
{ 0, 1, 2,
  4, 8, 16,
  32, 64, 128};

double *B = A+3;

sum_rows1(A, B, 3);
```

Value of B:

init: [4, 8, 16]

i = 0: [3, 8, 16]

i = 1: [3, 22, 16]

i = 2: [3, 22, 224]

- 代码在每次循环中更新 b[i]
- 必须考虑这些更新会影响程序行为的可能性

移除别名

```
/* Sum rows is of n X n matrix a
   and store in vector b */
void sum_rows1(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += a[i*n + j];
    }
}
```

```
/* Sum rows is of n X n matrix a
   and store in vector b */
void sum_rows2(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        double val = 0;
        for (j = 0; j < n; j++)
            val += a[i*n + j];
        b[i] = val;
    }
}
```

```
# sum_rows2 inner loop
.L10:
    addsd    (%rdi), %xmm0    # FP load + add
    addq     $8, %rdi
    cmpq     %rax, %rdi
    jne      .L10
```

- 无需存储中间结果

妨碍优化的因素：内存别名

■ 别名

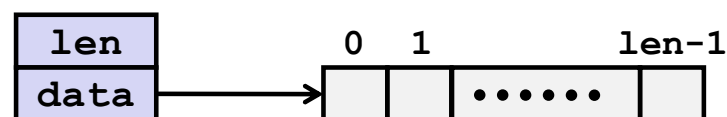
- 两个不同的内存引用指向同一个存储位置
- 在 C 语言中很容易出现这种情况
 - 因为 C 语言允许进行地址运算
 - 直接访问存储结构
- 习惯引入局部变量
 - 在循环中累加结果
 - 通过这种方式告诉编译器无需检查别名

利用指令级并行性

- 需要对现代处理器设计有基本了解
 - 硬件可以并行执行多条指令
- 性能受数据依赖关系限制
- 简单的转换可以显著提升性能
 - 编译器通常无法进行这些转换
 - 浮点运算没有结合律和分配律

基准测试程序：向量的数据类型

```
/* data structure for vectors */
typedef struct{
    size_t len;
    data_t *data;
} vec;
```



■ 数据类型

- 对 data_t 使用不同的声明方式
- int
- long
- float
- double

```
/* retrieve vector element
and store at val */
int get_vec_element
(*vec v, size_t idx, data_t *val)
{
    if (idx >= v->len)
        return 0;
    *val = v->data[idx];
    return 1;
}
```

基准测试程序使用的计算

```
void combinel(vec_ptr v, data_t *dest)
{
    long int i;
    *dest = IDENT;
    for (i = 0; i < vec_length(v); i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest OP val;
    }
}
```

计算向量元素的和或积

■ 数据类型

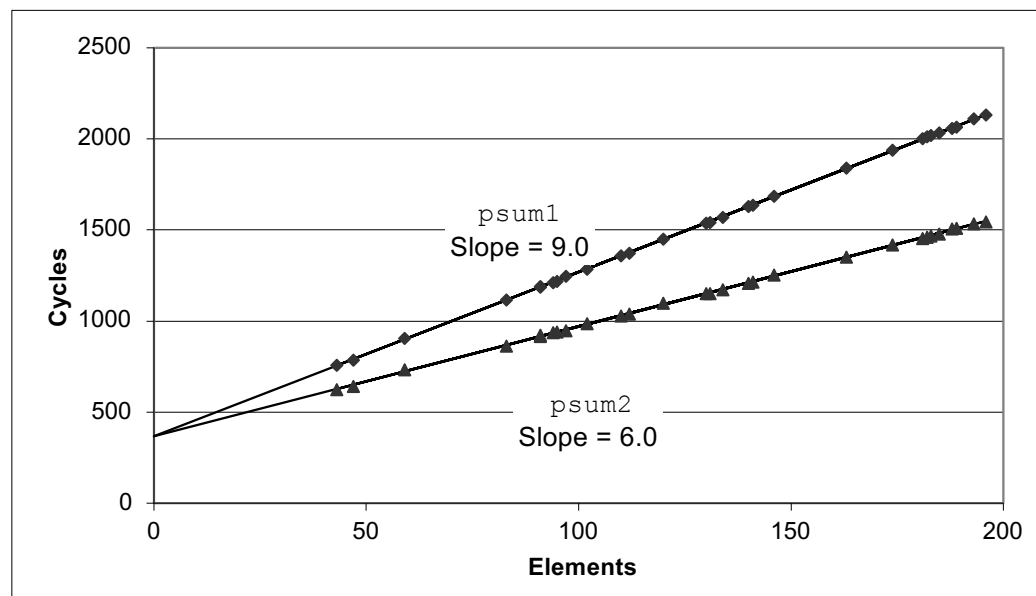
- 使用不同的类型声明 data_t
- int
- long
- float
- double

■ 运算操作

- 使用不同的运算符和初始值定义 OP 和 IDENT
- + / 0
- * / 1

每元素周期数 (Cycles Per Element, CPE)

- 方便地表示对向量或列表操作的程序的性能
- 长度为 n
- 在我们的例子里: **CPE = cycles per OP**
- **$T = CPE * n + \text{Overhead}$**
 - CPE 是该线性方程的斜率



基准测试程序性能

```
void combinel(vec_ptr v, data_t *dest)
{
    long int i;
    *dest = IDENT;
    for (i = 0; i < vec_length(v); i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest OP val;
    }
}
```

计算向量元素的和或积

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine1 unoptimized	22.68	20.02	19.98	20.18
Combine1 -O1	10.12	10.12	10.17	11.14

基础优化

```
void combine1(vec_ptr v, data_t *dest)
{
    long int i;
    *dest = IDENT;
    for (i = 0; i < vec_length(v); i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest OP val;
    }
}
```

```
void combine4(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    data_t *d = get_vec_start(v);
    data_t t = IDENT;
    for (i = 0; i < length; i++)
        t = t OP d[i];
    *dest = t;
}
```

- 将 `vec_length` 移出循环
- 避免每次循环的边界检查
- 在临时变量中累加

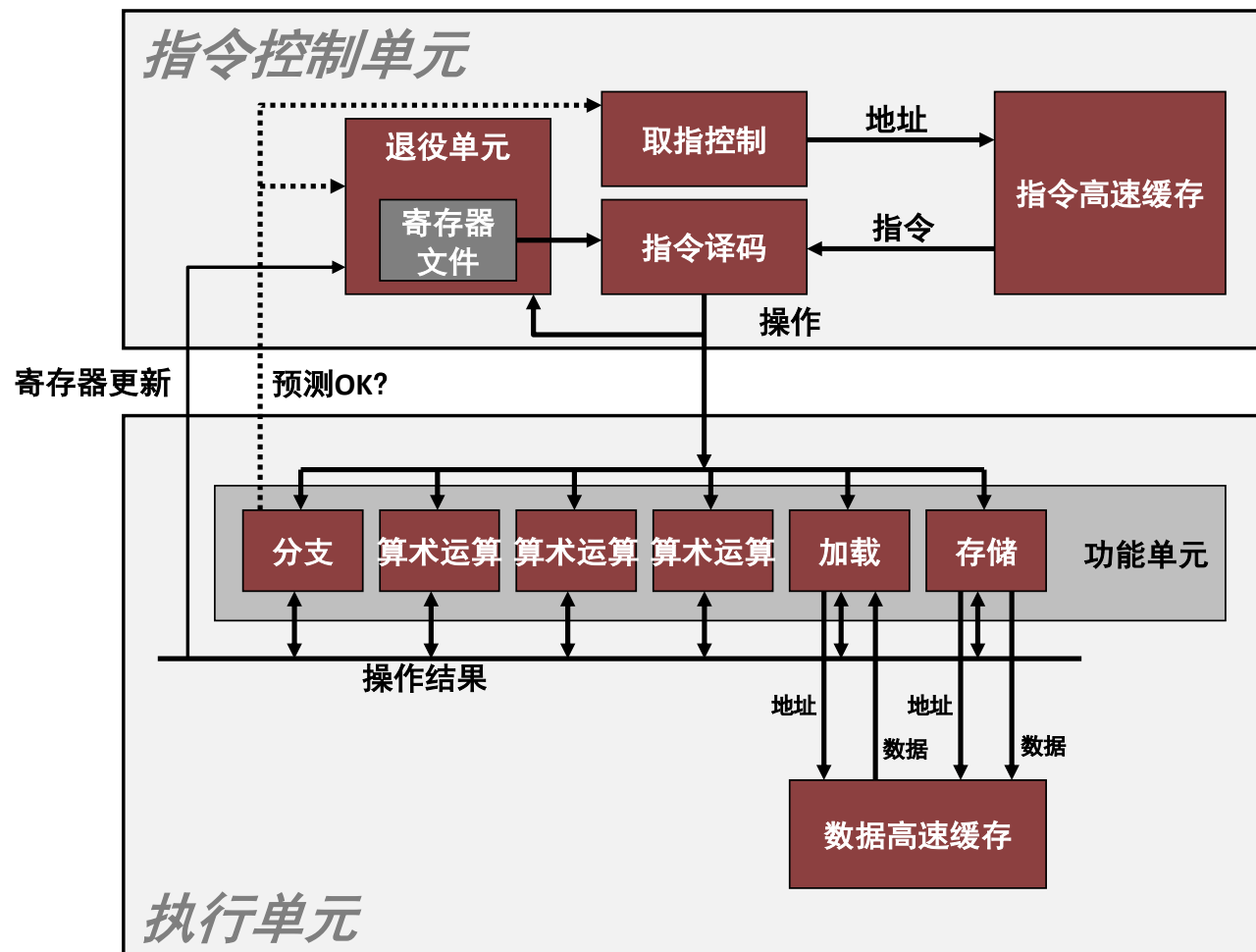
基础优化的效果

```
void combine4(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    data_t *d = get_vec_start(v);
    data_t t = IDENT;
    for (i = 0; i < length; i++)
        t = t OP d[i];
    *dest = t;
}
```

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine1 -01	10.12	10.12	10.17	11.14
Combine4	1.27	3.01	3.01	5.01

■ 消除循环中的开销来源

现代 CPU 设计

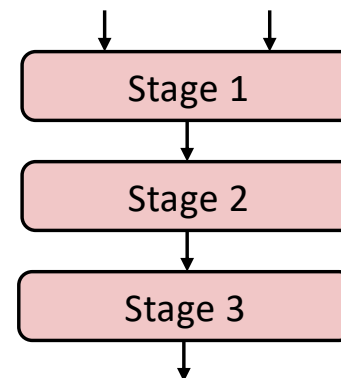


超标量处理器

- **定义：** 超标量处理器可以在一个时钟周期内发出和执行 **多条指令**。它从顺序指令流中获取指令，并通常采用动态调度的方式来执行这些指令。
- **优势：** 超标量处理器无需额外的编程工作即可利用大多数程序中的 **指令级并行性**。
- 大多数现代 **CPU** 是超标量处理器。
- **Intel:** 从 **Pentium (1993)** 起

流水线功能单元

```
long mult_eg(long a, long b, long c) {  
    long p1 = a * b;  
    long p2 = a * c;  
    long p3 = p1 * p2;  
    return p3;  
}
```



Time							
	1	2	3	4	5	6	7
Stage 1	a*b	a*c			p1*p2		
Stage 2		a*b	a*c			p1*p2	
Stage 3			a*b	a*c			p1*p2

- 分阶段处理计算
- 阶段间传递部分计算结果
- 一旦结果传递给第 **i+1** 阶段，第 **i** 阶段就可以开始新的计算
- 例如，尽管每个乘法操作需要 **3** 个周期，流水线设计可以在 **7** 个周期内完成 **3** 次乘法操作。

Haswell CPU

- 总计 8 个功能单元
- 多条指令可以并行执行

1. 整数操作, 浮点乘法, 整数和浮点除法, 分支操作
2. 整数操作, 浮点加法, 整数乘法, 浮点乘法
3. 加载操作, 地址计算
4. 加载操作, 地址计算
5. 存储操作
6. 整数操作
7. 整数操作, 分支操作
8. 存储地址计算

4个功能单元能执行整数操作

2个功能单元能执行加载操作

2个功能单元能执行浮点乘法

1个功能单元能执行浮点加法操作

Haswell CPU

- 有些指令需要超过 1 个周期，但可以进行流水线处理

<i>Instruction</i>	<i>Latency</i>	<i>Cycles/Issue</i>	<i>Capacity</i>
Integer Add	1	1	4
Integer Multiply	3	1	1
Integer/Long Divide	3-30	3-30	1
Single/Double FP Add	3	1	1
Single/Double FP Multiply	5	1	2
Single/Double FP Divide	3-15	3-15	1

latency 延迟：完成运算所需要的总时间

issue time 发射时间I：两个连续的同类型运算之间需要的最小时钟周期数。

发射时间为1的功能单元被称为完全流水线化的（**fully pipelined**）。

capacity 容量C：能够执行该运算的功能单元数量

吞吐量： C/I

x86-64 Combine4 编译

■ 内部循环（整数乘法）

```
.L519:                # Loop:
    imull  (%rax,%rdx,4), %ecx  # t = t * d[i]
    addq   $1, %rdx            # i++
    cmpq   %rdx, %rbp          # Compare length:i
    jg     .L519               # If >, goto Loop
```

■ 内部循环（双精度乘法）

```
.L25:
    vmulsd (%rdx), %xmm0, %xmm0 # Multiply acc by data[i]
    addq   $8, %rdx             # Increment data+i
    cmpq   %rax, %rdx           # Compare to data+length
    jne    .L25                 # If !=, goto loop
```

```
void combine4(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    data_t *d = get_vec_start(v);
    data_t t = IDENT;
    for (i = 0; i < length; i++)
        t = t OP d[i];
    *dest = t;
}
```

Combine4 的图形化表示

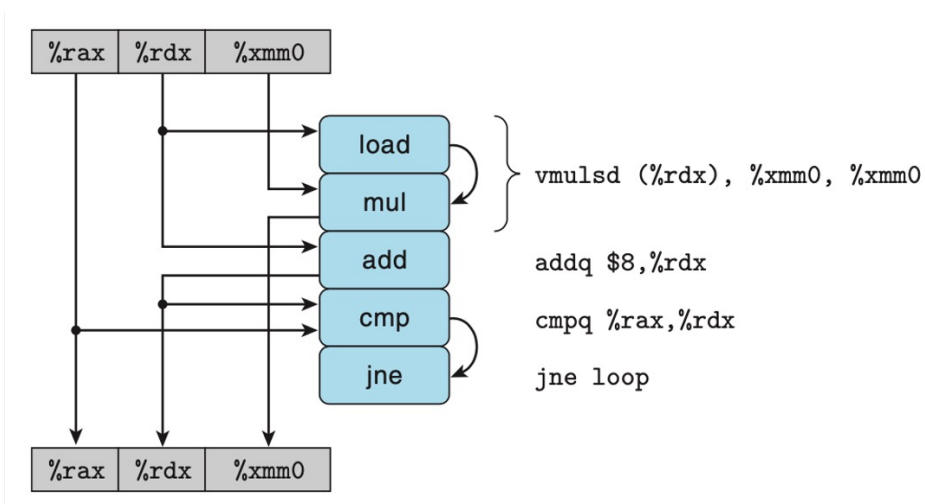
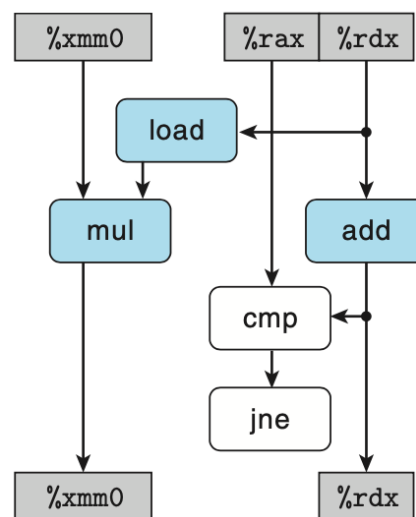
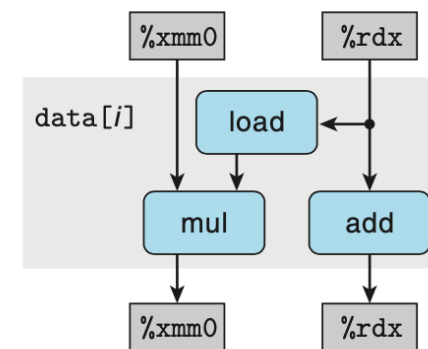


图 5.13 combine4 的内循环代码的图形化表示



(a)



(b)

图 5.14 将 combine4 的操作抽象成数据流图

Combine4 的图形化表示

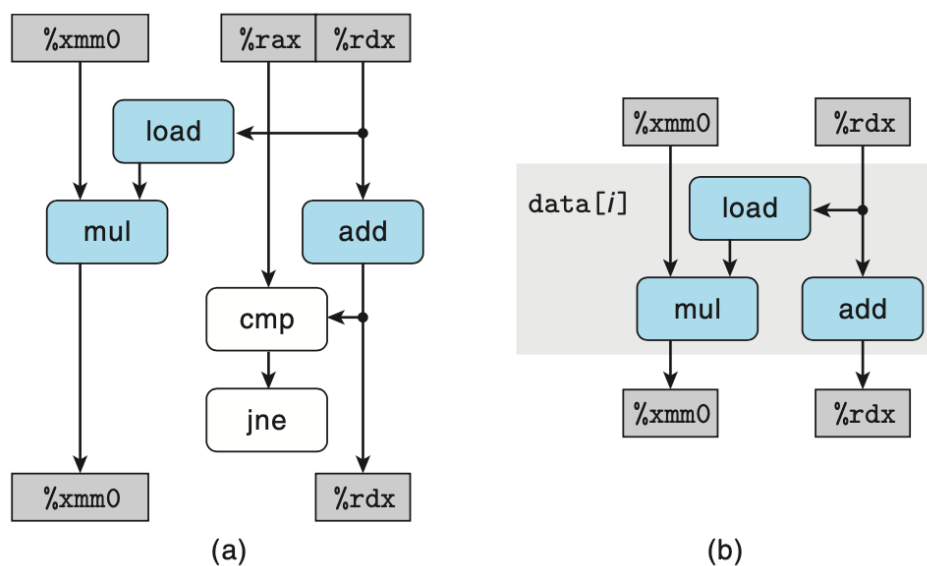


图 5.14 将 combine4 的操作抽象成数据流图

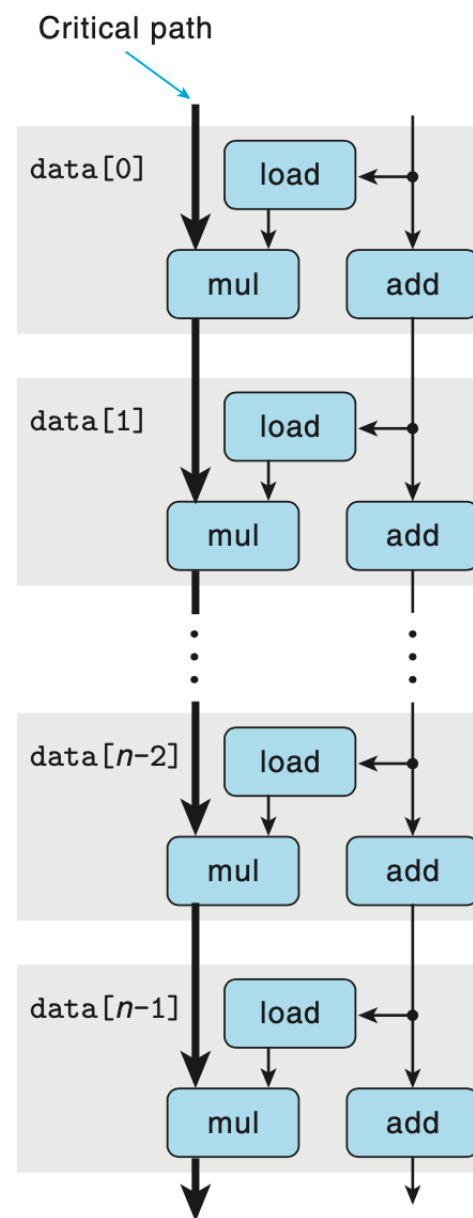
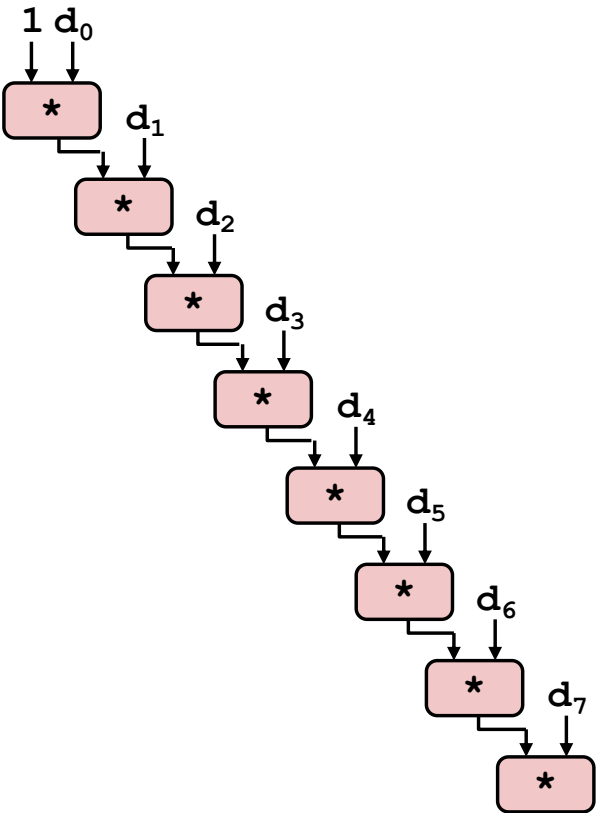


图 5.15 combine4 的内循环的 n 次迭代的数据流表示

Combine4 = 串行计算 (OP = *)



- 计算过程 (长度为 8)
(((((((1 * d[0]) * d[1]) * d[2]) * d[3]) * d[4]) * d[5]) * d[6]) * d[7])
- 顺序依赖
 - 性能受限于 * 操作的延迟

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Latency Bound	1.00	3.00	3.00	5.00

循环展开 (2x1)

```
void unroll2a_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x = (x OP d[i]) OP d[i+1];
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x = x OP d[i];
    }
    *dest = x;
}
```

- 每次迭代执行 2 倍以上的有用工作

循环展开的效果

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Latency Bound	1.00	3.00	3.00	5.00

- 对整数加法有帮助
 - 达到延迟上限 (latency bound)
- 其他操作未改善 *Why?*
 - 依然存在顺序依赖。

```
x = (x OP d[i]) OP d[i+1];
```

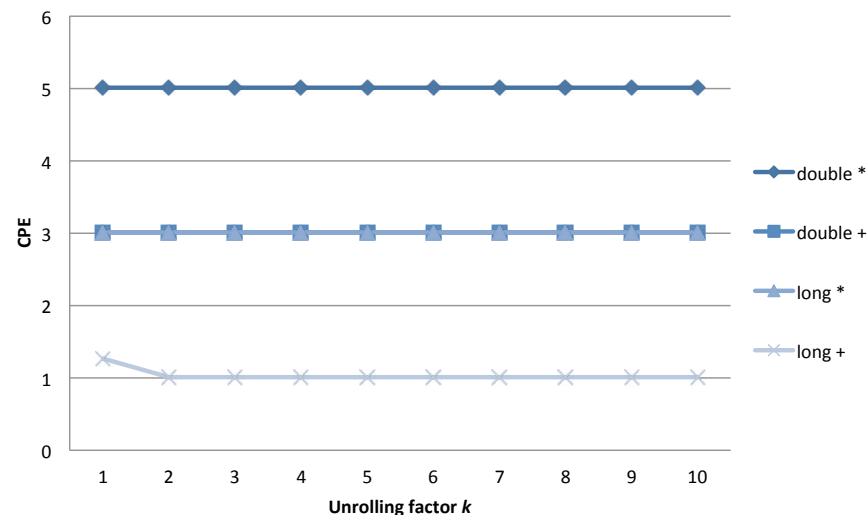


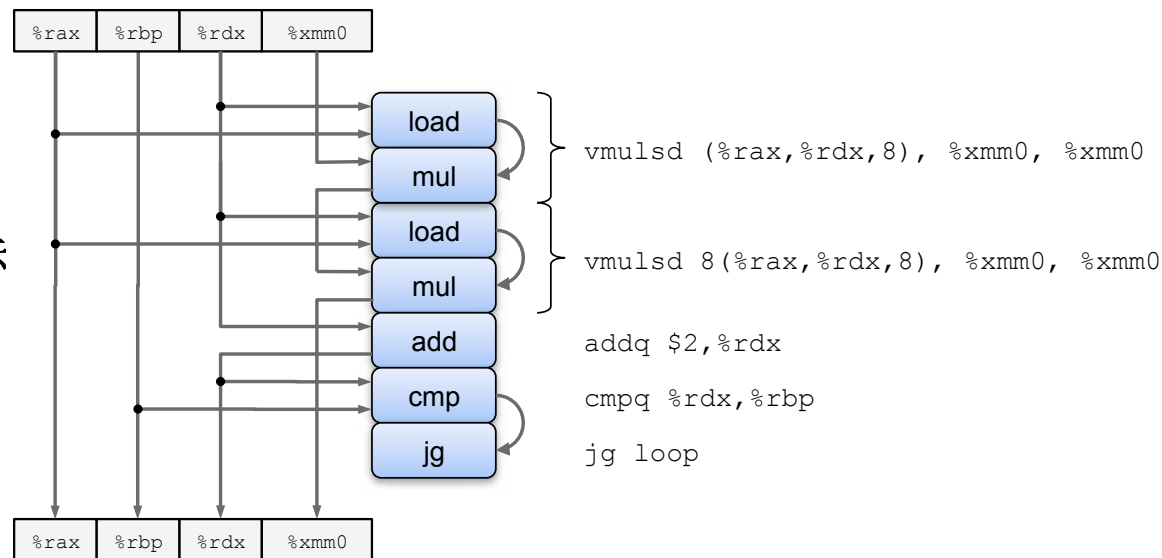
图 5.17 不同程度 $k \times 1$ 循环展开的 CPE 性能。这种变换只改进了整数加法性能

循环展开: Combine5

Inner loop of combine5. data_t = double, OP = * i in %rdx, data %rax, limit in %rbp, acc in %xmm0

```
.L35:                                #loop:
    vmulsd (%rax,%rdx,8), %xmm0, %xmm0  #Multiply acc by data[i]
    vmulsd 8(%rax,%rdx,8), %xmm0, %xmm0 #Multiply acc by data[i+1]
    addq    $2, %rdx                  #Increment i by 2
    cmpq    %rdx, %rbp                #Compare to limit:i
    jg      .L35                      #If >, goto loop
```

图 5.18 combine5 的内循环代码的图形化表示



Combine5 的图形化表示

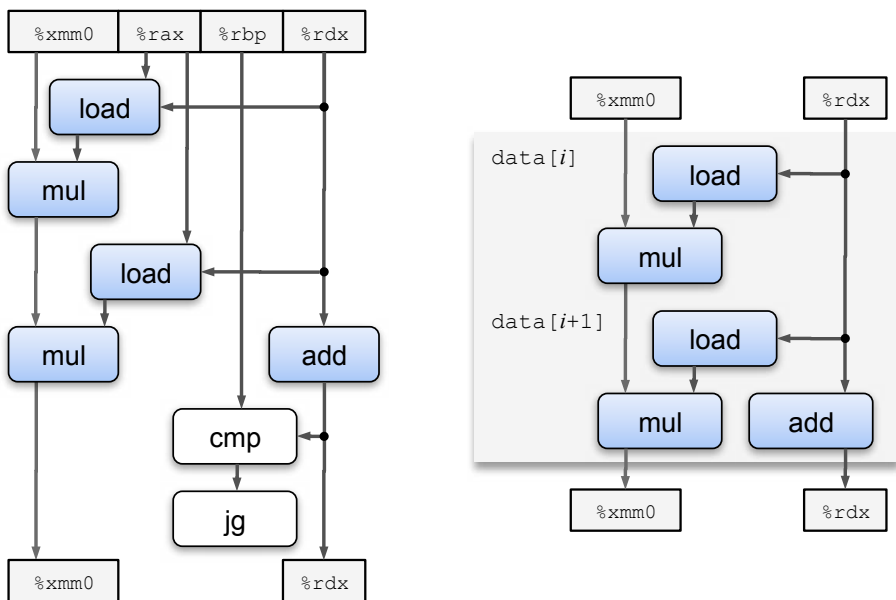
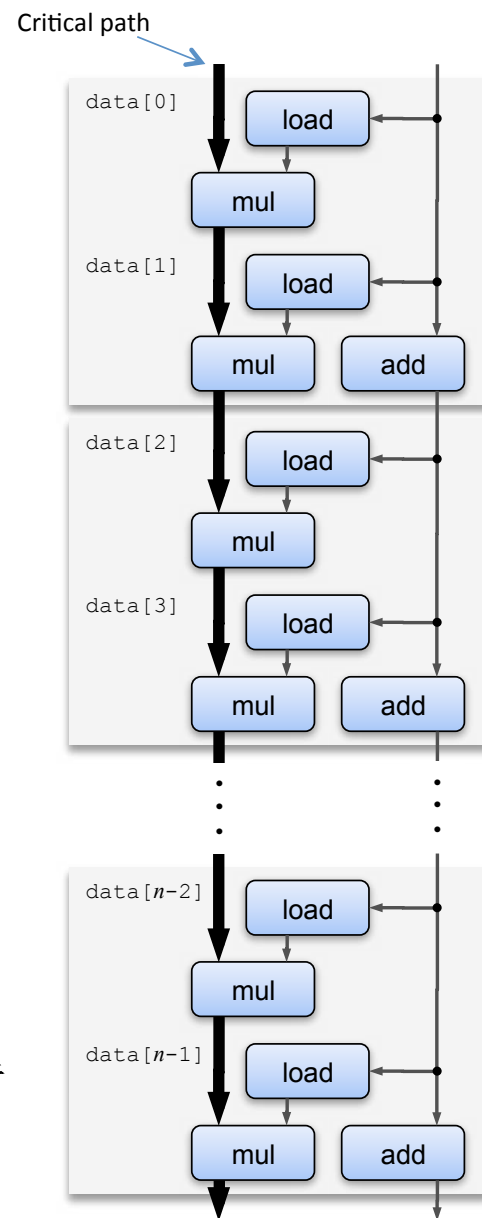


图 5.19 将 combine5 的操作抽象成数据流图

图 5.20 combine5 对一个长度为 n 的向量进行操作的数据流表示



循环展开与重新结合(2x1a)

```
void unroll2aa_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x = x OP (d[i] OP d[i+1]);
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x = x OP d[i];
    }
    *dest = x;
}
```

与之前相比

$x = (x \text{ OP } d[i]) \text{ OP } d[i+1];$

- 这个优化会改变计算结果吗？
- 是的，对于浮点运算。 *Why?*

重新结合的效果

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
Latency Bound	1.00	3.00	3.00	5.00
Throughput Bound	0.50	1.00	1.00	0.50

- 对于整数*, 浮点+, 浮点*, 几乎是两倍的提升

- 原因: 打破顺序依赖

```
x = x OP (d[i] OP d[i+1]);
```

- 为什么?

1 func. units for int *
2 func. units for load

2 func. units for FP *
2 func. units for load

1 func. units for FP +
2 func. units for load

4 func. units for int +
2 func. units for load

Combine7 的图形化表示

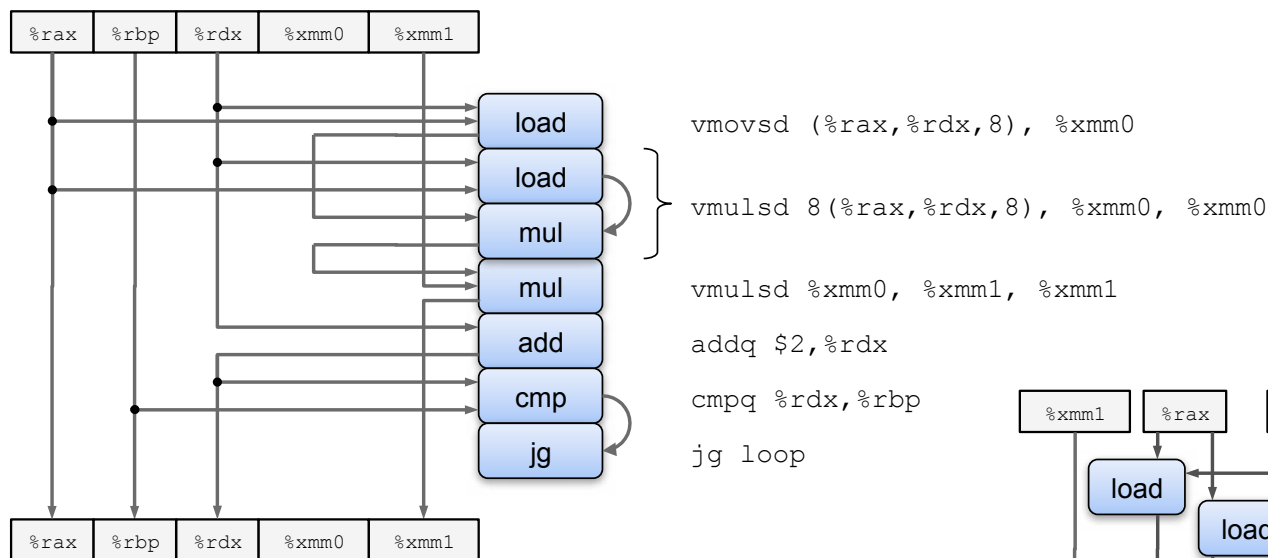


图 5.27 combine7 的内循环代码的图形化表示

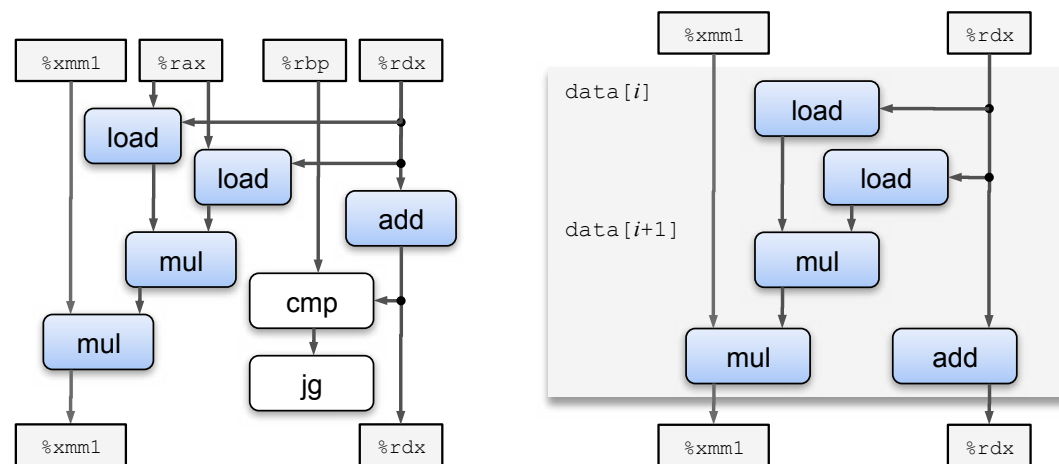
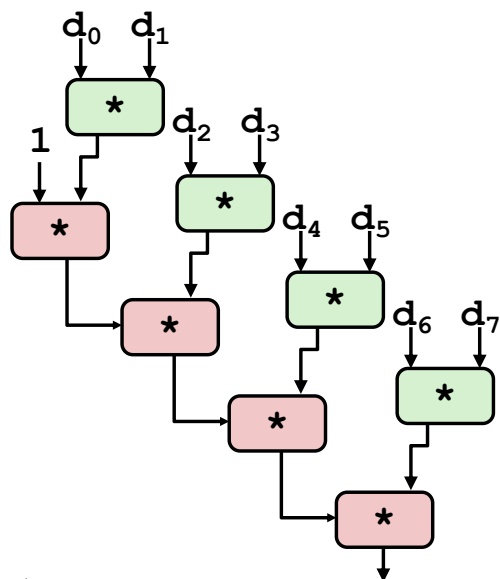


图 5.28 将 combine5 的操作抽象成数据流图

重新结合计算

$x = x \text{ OP } (d[i] \text{ OP } d[i+1]);$



■ 什么改变了:

- 下一次迭代中的操作可以提前开始（没有依赖）

■ 整体性能

- N 个元素，每个 op 有 D 个周期的操作延迟
- $(N/2+1)*D$ cycles:
 $CPE = D/2$

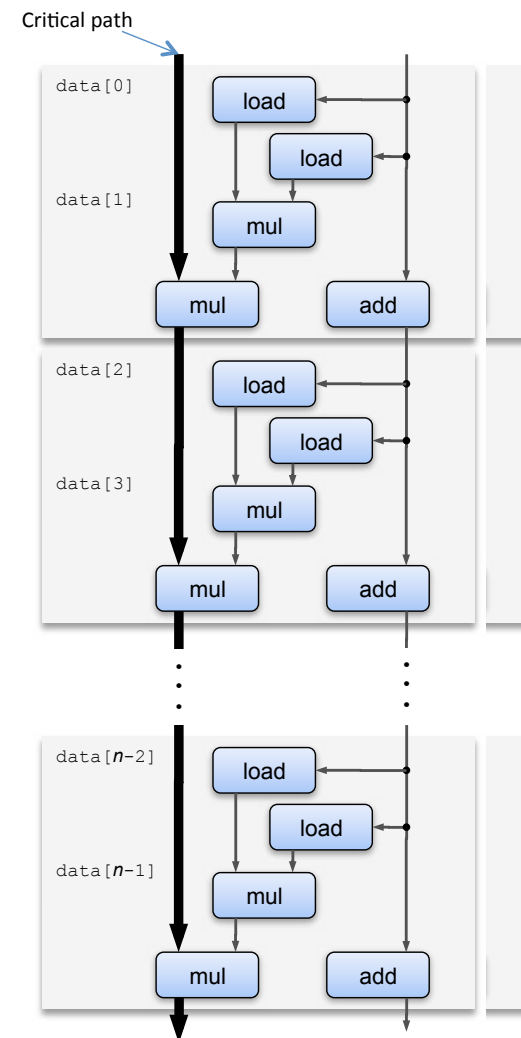


图 5.29 combine7 对一个长度为 n 的向量进行操作的数据流表示。我们只有一条关键路径，他只包含 $n/2$ 个操作

重新结合的效果

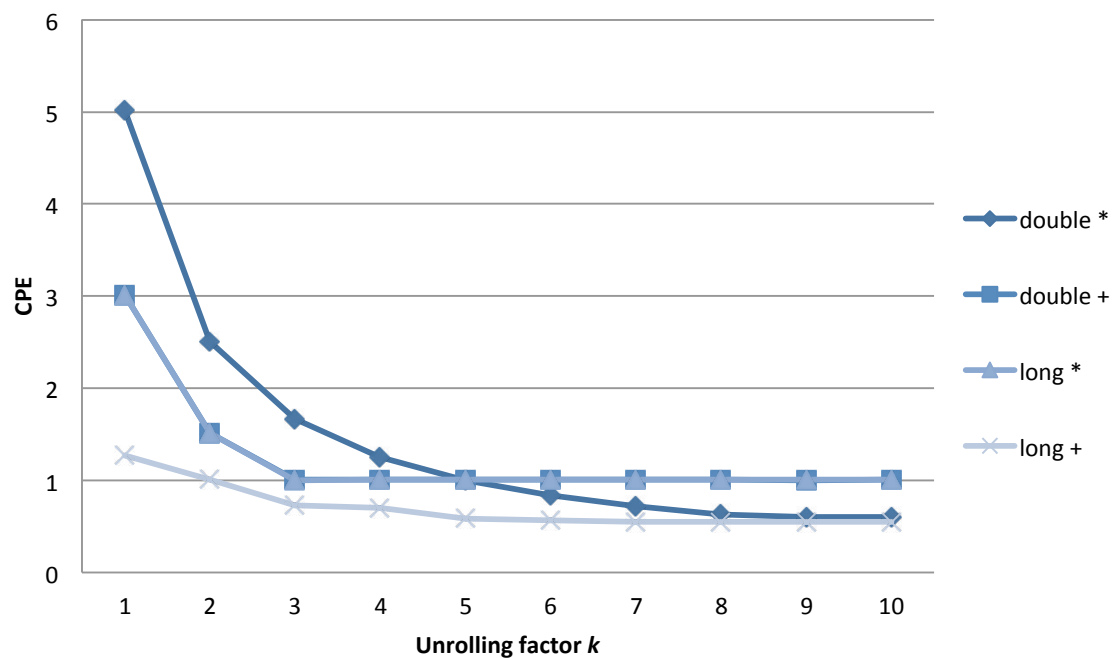


图 5.30 $k \times 1a$ 循环展开的 CPE 性能。在这种变换下，所有的 CPE 都有所改进，几乎达到了他们的吞吐量界限

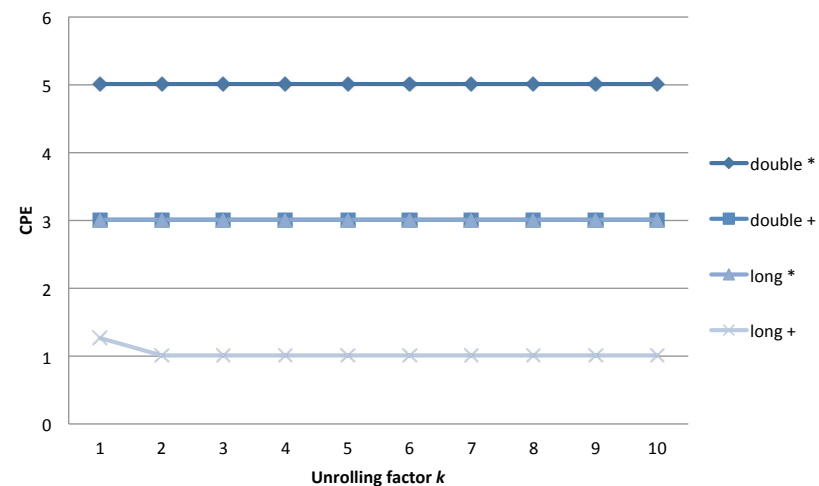


图 5.17 不同程度 $k \times 1$ 循环展开的 CPE 性能。这种变换只改进了整数加法性能

循环展开与独立累加器(2x2)

```
void unroll2a_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x0 = IDENT;
    data_t x1 = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x0 = x0 OP d[i];
        x1 = x1 OP d[i+1];
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x0 = x0 OP d[i];
    }
    *dest = x0 OP x1;
}
```

■ 重新结合的不同形式

独立累加器的效果

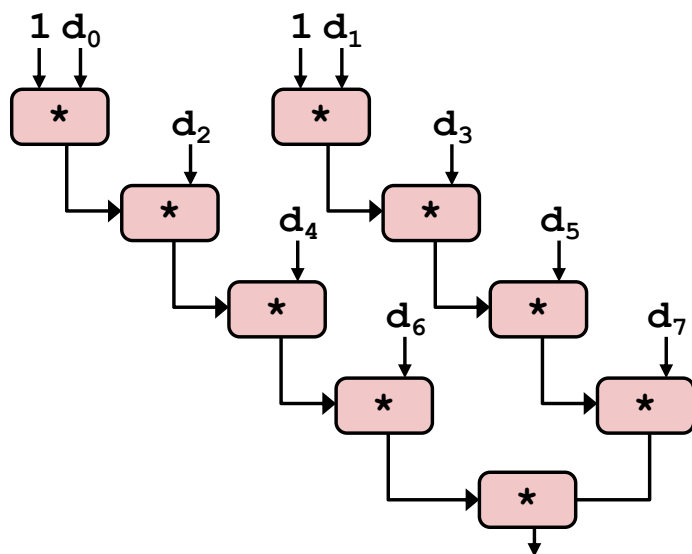
Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
Unroll 2x2	0.81	1.51	1.51	2.51
Latency Bound	1.00	3.00	3.00	5.00
Throughput Bound	0.50	1.00	1.00	0.50

```
x0 = x0 OP d[i];
x1 = x1 OP d[i+1];
```

- Int + 使用两个load单元
- Int *, FP +, FP * 得到两倍加速（相比 2x1 展开）

独立累加器

```
x0 = x0 OP d[i];  
x1 = x1 OP d[i+1];
```



■ 什么改变了？

- 两个独立的操作“流”

■ 整体性能

- N 个元素，每个 op 有 D 个周期的操作延迟
- Should be $(N/2+1)*D$ cycles:
CPE = D/2
- CPE 符合预期！

下一步？

展开和累加

■ 思路

- 可以展开到任意程度 L
- 可以并行累加 K 个结果
- L 必须是 K 的倍数

■ 限制

- 收益递减
 - 无法突破执行单元的吞吐量限制
- 对于较短的长度来说开销较大
 - 按顺序完成迭代

展开和累加: Double *

■ 情况

- Intel Haswell
- 双精度浮点数乘法
- 延迟界限: 5.00 个周期
- 吞吐量界限: 0.50 个周期

Accumulators	FP *	Unrolling Factor L							
	K	1	2	3	4	6	8	10	12
	1	5.01	5.01	5.01	5.01	5.01	5.01	5.01	
	2		2.51		2.51		2.51		
	3			1.67					
	4				1.25		1.26		
	6					0.84			0.88
	8						0.63		
	10							0.51	
	12								0.52

展开和累加: Int +

■ 情况

- Intel Haswell
- 整数加法
- 延迟界限: 1.00 个周期
- 吞吐量界限: 0.50 个周期

Accumulators	FP *	Unrolling Factor L							
	K	1	2	3	4	6	8	10	12
	1	1.27	1.01	1.01	1.01	1.01	1.01	1.01	
	2		0.81		0.69		0.54		
	3			0.74					
	4				0.69		1.24		
	6					0.56			0.56
	8						0.54		
	10							0.54	
	12								0.56

可实现的性能

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Best	0.54	1.01	1.01	0.52
Latency Bound	1.00	3.00	3.00	5.00
Throughput Bound	0.50	1.00	1.00	0.50

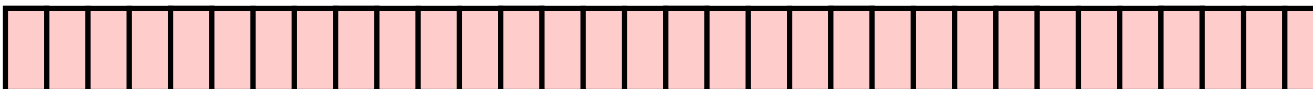
- 仅受功能单元吞吐量的限制
- 与原始的未优化代码相比，性能提高了 **42** 倍

使用 AVX2 编程

YMM 寄存器

■ 总计 16 个寄存器，每个 32 字节

■ 32 个单字节整数



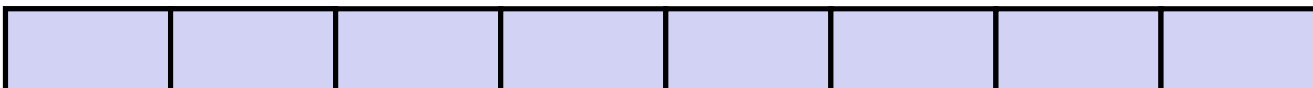
■ 16 个 16 位整数



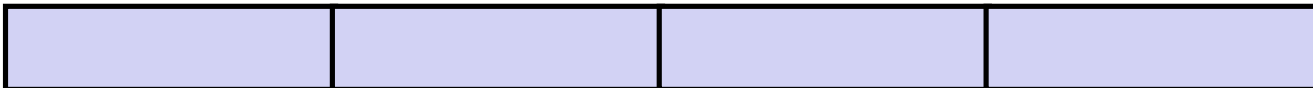
■ 8 个 32 位整数



■ 8 个单精度浮点数



■ 4 个双精度浮点数



■ 1 个单精度浮点数



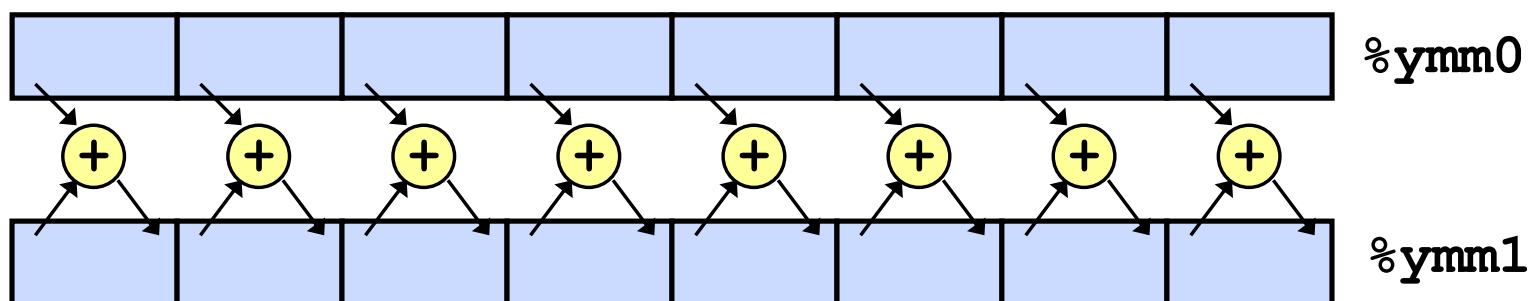
■ 1 个双精度浮点数



SIMD 操作

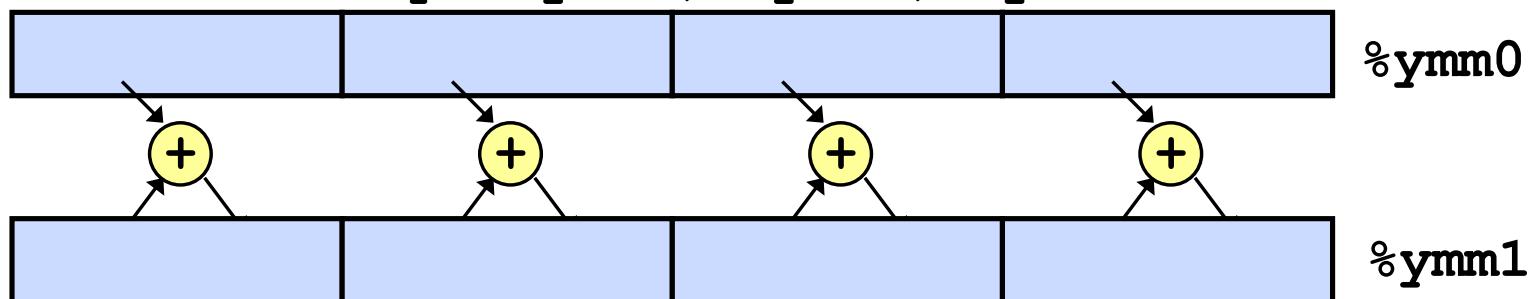
■ SIMD 操作：单精度

`vaddps %ymm0, %ymm1, %ymm1`



■ SIMD 操作：双精度

`vaddpd %ymm0, %ymm1, %ymm1`



使用向量指令

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Scalar Best	0.54	1.01	1.01	0.52
Vector Best	0.06	0.24	0.25	0.16
Latency Bound	0.50	3.00	3.00	5.00
Throughput Bound	0.50	1.00	1.00	0.50
Vec Throughput Bound	0.06	0.12	0.25	0.12

■ 利用 **AVX** 指令

- 对多个数据元素进行并行操作
- 参见 **CS:APP** 网页上的网络旁注 **OPT:SIMD**

There's plenty of room at the Top: What will drive computer performance after Moore's law?

Leiserson et al., Science 368, 1079 (2020), 5 June 2020

ming language. Here is the four-line kernel of
the Python 2 code for matrix-multiplication:

```
for i in xrange(4096):  
    for j in xrange(4096):  
        for k in xrange(4096):  
            C[i][j] += A[i][k] * B[k][j]
```

Table 1. Speedups from performance engineering a program that multiplies two 4096-by-4096 matrices. Each version represents a successive refinement of the original Python code. "Running time" is the running time of the version. "GFLOPS" is the billions of 64-bit floating-point operations per second that the version executes. "Absolute speedup" is time relative to Python, and "relative speedup," which we show with an additional digit of precision, is time relative to the preceding line. "Fraction of peak" is GFLOPS relative to the computer's peak 835 GFLOPS. See Methods for more details.

Version	Implementation	Running time (s)	GFLOPS	Absolute speedup	Relative speedup	Fraction of peak (%)
1	Python	25,552.48	0.005	1	—	0.00
2	Java	2,372.68	0.058	11	10.8	0.01
3	C	542.67	0.253	47	4.4	0.03
4	Parallel loops	69.80	1.969	366	7.8	0.24
5	Parallel divide and conquer	3.80	36.180	6,727	18.4	4.33
6	plus vectorization	1.10	124.914	23,224	3.5	14.96
7	plus AVX intrinsics	0.41	337.812	62,806	2.7	40.45

一些限制因素

■ 寄存器溢出

Function	Page	Method	Integer		Floating point	
			+	*	+	*
combine6	537					
		10 × 10 unrolling	0.55	1.00	1.01	0.52
		20 × 20 unrolling	0.83	1.03	1.02	0.68
Throughput bound			0.50	1.00	1.00	0.50

寄存器溢出

■ 10 × 10 循环展开的内循环代码

```
Updating of accumulator acc0 in 10 x 10 urolling acc0 *= data[i]  
vmulsd (%rdx), %xmm0, %xmm0
```

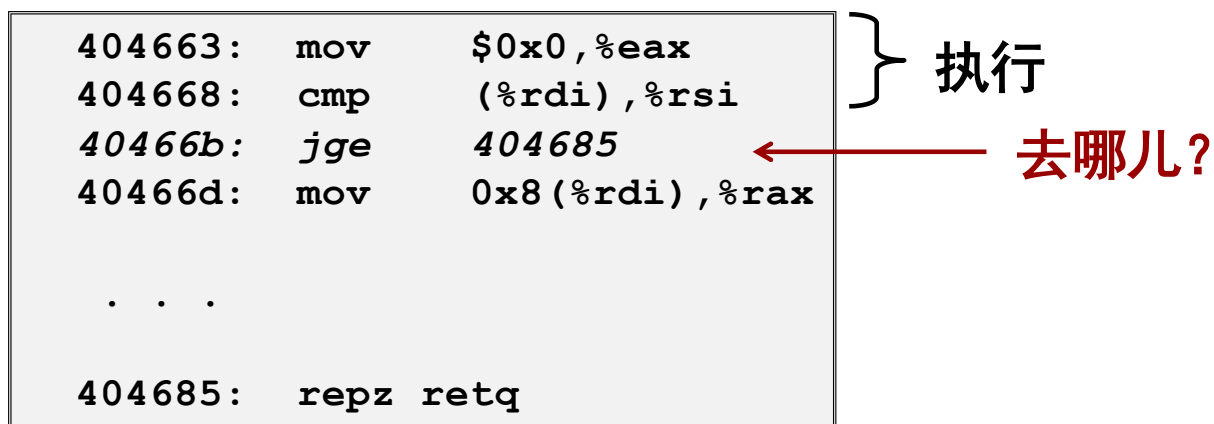
■ 20 × 20 循环展开的内循环代码

```
Updating of accumulator acc0 in 20 x 20 unrolling  
vmovsd 40(%rsp), %xmm0  
vmulsd (%rdx), %xmm0, %xmm0  
vmovsd %xmm0, 40(%rsp)
```

分支?

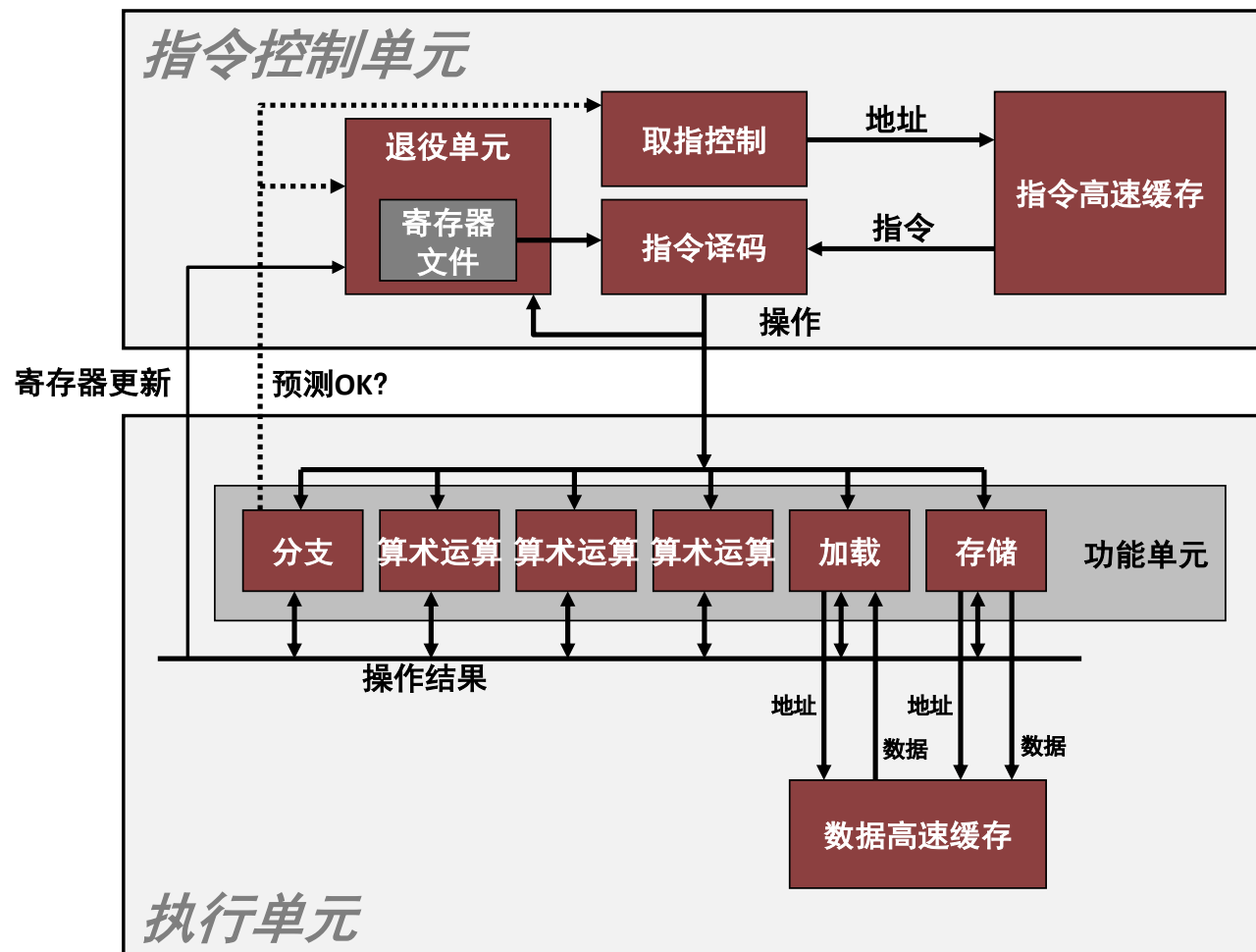
■ 挑战

- 指令控制单元必须领先于执行单元，以生成足够的操作来使执行单元保持忙碌



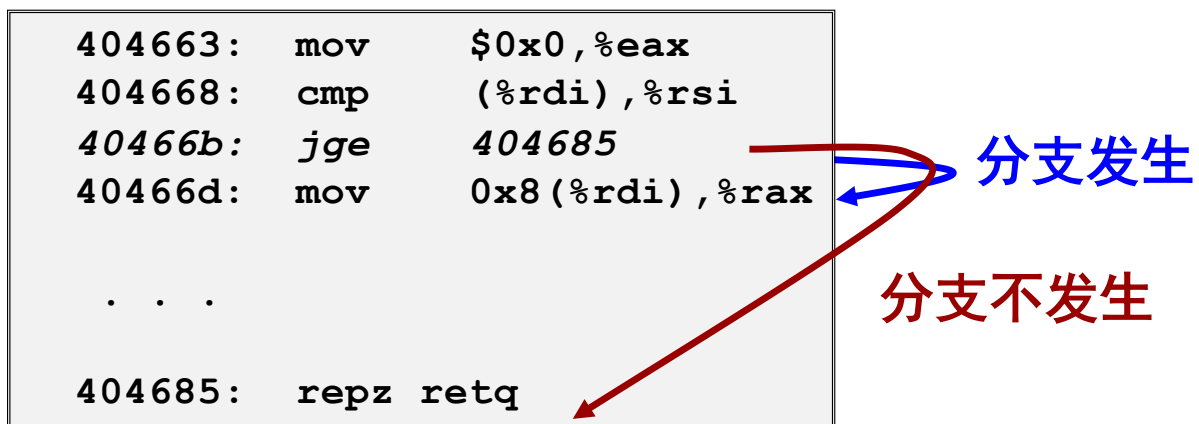
- 当遇到条件分支时，无法可靠地确定下一步从哪里继续获取指令

现代 CPU 设计



分支结果

- 遇到条件分支时，无法确定下一步从哪里继续获取指令
 - 分支发生：将控制权转移到分支目标
 - 分支不发生：按顺序继续执行下一条指令
- 在分支/整数单元确定结果之前无法解决



分支预测

■ 思路

- 猜测分支的走向
- 开始执行预测位置的指令
 - 但不修改寄存器或内存数据

```
404663:  mov    $0x0,%eax
404668:  cmp    (%rdi),%rsi
40466b:  jge    404685
40466d:  mov    0x8(%rdi),%rax

. . .

404685:  repz   retq
```

预测分支发生

} 开始执行

在循环中进行分支预测

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 98
```

假设

vector length = 100

预测分支发生 (OK)

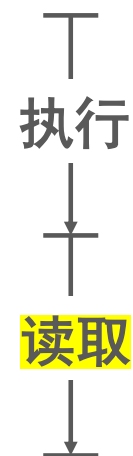
```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 99
```

预测分支不发生
(Oops)

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 100
```

读取无效地址

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 101
```



分支预测错误

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 98
```

假设

vector length = 100

预测分支发生 (OK)

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 99
```

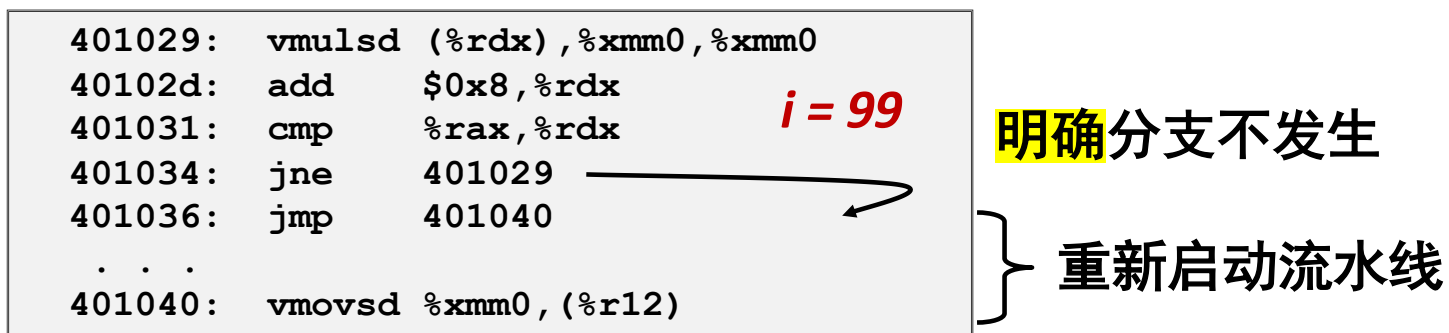
预测分支不发生
(Oops)

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 100
```

无效

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 101
```

分支预测错误恢复



■ 性能代价

- 在现代处理器上需要多个时钟周期
- 可能成为主要的性能限制因素

Code Example

```
#include <algorithm>
#include <ctime>
#include <iostream>

int main()
{
    // Generate data
    const unsigned arraySize = 32768;
    int data[arraySize];

    for (unsigned c = 0; c < arraySize; ++c)
        data[c] = std::rand() % 256;

    // !!! With this, the next loop runs faster.
    std::sort(data, data + arraySize);
```

	有排序版本	无排序版本
g++ -O0	5.652633	16.714600
g++ -Ofast	0.606013	0.604250

```
// Test
clock_t start = clock();
long long sum = 0;
for (unsigned i = 0; i < 100000; ++i)
{
    for (unsigned c = 0; c < arraySize; ++c)
    { // Primary loop.
        if (data[c] >= 128)
            sum += data[c];
    }
}

double elapsedTime = static_cast<double>(clock()-start) /
CLOCKS_PER_SEC;

std::cout << elapsedTime << '\n';
std::cout << "sum = " << sum << '\n';
}
```

不要过分关心可预测的分支

■ Combine2

- 将 `vec_length` 的调用移出循环测试

■ Combine3

- 将函数 `get_vec_element` 移出函数内循环

Function	Page	Method	Integer		Floating point	
			+	*	+	*
<code>combine2</code>	509	Move <code>vec_length</code>	7.02	9.03	9.02	11.03
<code>combine3</code>	513	Direct data access	7.17	9.02	9.02	11.03

```

1  /* Move call to vec_length out of loop */
2  void combine2(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6
7      *dest = IDENT;
8      for (i = 0; i < length; i++) {
9          data_t val;
10         get_vec_element(v, i, &val);
11         *dest = *dest OP val;
12     }
13 }

```

图 5.6 改进循环测试的效率。

```

1  /* Direct access to vector data */
2  void combine3(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t *data = get_vec_start(v);
7
8      *dest = IDENT;
9      for (i = 0; i < length; i++) {
10         *dest = *dest OP data[i];
11     }
12 }

```

图 5.9 消除循环中的函数调用。

```

int get_vec_element(vec_ptr v, long index, data_t *dest)
{
    if (index < 0 || index >= v->len)
        return 0;
    *dest = v->data[index];
    return 1;
}

```

使用内联函数访问数据元素

```
1  /* Include bounds check in loop */
2  void combine4b(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t acc = IDENT;
7
8      for (i = 0; i < length; i++) {
9          if (i >= 0 && i < v->len) {
10             acc = acc OP v->data[i];
11         }
12     }
13     *dest = acc;
14 }
```

Function	Page	Method	Integer		Floating point	
			+	*	+	*
combine4	515	No bounds checking	1.27	3.01	3.01	5.01
combine4b	515	Bounds checking	2.02	3.01	3.01	5.01

书写适合用条件传送实现的代码

```
1. void minmax1(long a[], long b[], long n) {  
2.     long i;  
3.     for (i = 0; i < n; i++) {  
4.         if (a[i] > b[i]) {  
5.             long t = a[i];  
6.             a[i] = b[i];  
7.             b[i] = t;  
8.         }  
9.     }  
10. }
```

- 对于随机数据，**CPE 13.5**
- 对于可预测数据，**CPE 2.5–3.5**，表示预测错误惩罚大约在 **20** 个周期

书写适合用条件传送实现的代码

```
1. void minmax1(long a[], long b[], long n) {  
2.     long i;  
3.     for (i = 0; i < n; i++) {  
4.         long min = a[i] < b[i] ? a[i] : b[i];  
5.         long max = a[i] < b[i] ? b[i] : a[i];  
6.         a[i] = min;  
7.         b[i] = max;  
8.     }  
9. }
```

■ **CPE 约为 4.0**，无论数据是任意的还是可预测的

获得高性能

- 良好的编译器和标志

- 避免不合理的做法

- 注意隐藏的算法带来的低效率
- 编写对编译器友好的代码
 - 注意妨碍优化的因素：过程调用和内存别名
- 仔细检查最内层循环（大部分工作在此进行）

- 为机器调优代码

- 利用指令级并行性
- 避免不可预测的分支
- 使代码适应缓存